

Practical Data Structures for Mode and Chromatic k -NN Queries

Geert-Jan Giezeman*

Erwin Glazenburg*

Frank Staals*

Abstract. Let P be a set of colored points in $\mathbb{R}^d$. We wish to store them such that, given a query point $q \in \mathbb{R}^d$ and an integer k , we can quickly answer *chromatic k -nearest neighbor (k -NN) queries*, that is, compute the most frequent color among the k points from P nearest to q . We are interested in data structures that can answer such queries efficiently, both in theory but also in practice. We focus on the L_∞ and L_1 distance measure. Van der Horst, Löffler, and Staals presented a linear space data structure that can answer such queries in roughly $\tilde{O}(n^{1-1/(d+1)})$ time. The key ingredient of this data structure is a cutting of piecewise linear functions in $\mathbb{R}^3$. We show that a simpler data structure by Chan, Durocher, Larsen, Morrison, and Wilkinson can be adapted to achieve the same complexities, and experimentally investigate how well this simpler data structure performs in practice. Additionally, we implement the cutting based approach to answer chromatic k -NN queries in $\mathbb{R}^1$ and compare it to (an appropriately adapted version of) our simplified data structure, as well as other existing methods. These experiments show that the cutting based structure indeed performs significantly worse than the simpler datastructure, despite having equal worst-case asymptotic complexities.

*Department of Information and Computing Sciences,
Utrecht University, The Netherlands (g.j.giezeman@uu.nl,
e.p.glazenburg@uu.nl, f.staals@uu.nl)

1 Introduction. Let P be a set of n points in $\mathbb{R}^d$, with $d = O(1)$, each of which has a color from $\{1.. \phi\}$. See Figure 1.1. We wish to store these points so that we can efficiently answer *chromatic k -nearest neighbor queries*: given a query point $q \in \mathbb{R}^d$ and a positive natural number $k \in \mathbb{N}$, report a *mode color* among the k -nearest neighbors of q , i.e. a color that appears the most frequently among the k points from P closest to q . This problem is motivated by classification problems; indeed answering a query corresponds to classifying a point using a k -NN classifier [4, 14]. The typical approach to answering such queries is to store the points in some sort of k -nearest neighbor reporting data structure, and iterate through the set $N_k(q) \subseteq P$ of k points closest to the query point q , while maintaining the mode color. Such an approach requires at least $\Omega(k)$ time. Van der Horst, Löffler, and Staals [16] showed that such linear dependency on k is unnecessary, and instead provide linear space data structures whose query time is sublinear, and independent of k . For example, using the L_2 metric in $d = 2$ their data structure achieves $\tilde{O}(n^{5/6})$ query time (where $\tilde{O}$ hides logarithmic factors in n). Similarly, for points in $\mathbb{R}^1$, such queries can be answered in $\tilde{O}(\sqrt{n})$ time while using linear space. This is almost tight (up to logarithmic factors and depending on the exact model of computation) in the worst case [2]. While these bounds may seem high, for classification queries it may indeed be desirable to include a large subset of the points, and thus k could be fairly large.

We would like to establish how applicable these approaches are, not just in theory, but also in practice. This is the main research question in this paper.

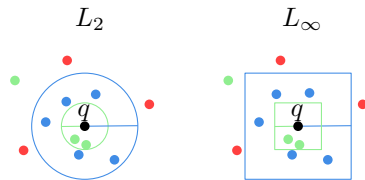


Figure 1.1: A small colored point set and query point q ; left with the L_2 distance measure, and right with the L_∞ distance measure. For both $k = 2$ would yield green, while $k = 7$ yields blue.

The problem setting. We will consider one of the “easiest” settings for the data structures: points in $d = 1$ and $d = 2$ dimensions, and using the $L_\infty(p, q) = \max_{i=1}^d |p_i - q_i|$ metric. If even in these settings these data structures cannot out-perform more naive strategies there seems little hope for more realistic scenarios (higher dimensions and more general distance measures). The key component in these data structures is a data structure for *range mode queries*, that is,

reporting the mode color among the points in a query range. We actually show that combining ideas from both van der Horst, Löffler, and Staals [16] and Chan et al. [2] we obtain a simple $O(n)$ space data structure, that can answer 2D range mode queries with axis aligned squares —and thus answer our chromatic k -NN queries with the L_∞ metric— in $\tilde{O}(n^{2/3})$ time. This matches the bound from van der Horst, Löffler, and Staals, but is arguably simpler than their data structure. This approach generalizes to any constant dimension d , and any distance measure whose unit disk is a constant complexity polytope, such as L_1 . In our chosen setting, answering range mode queries is related to answering type-2 color counting queries [7, 8, 6], computing heavy hitters [10], and answering top- k queries [18, 13]. Conceivably, our ideas may transfer to some of these settings.

Research Questions. To answer our main research question, determining how applicable these mode-finding data structures are in practice, we identify several important sub-questions:

1. Several of the data structures have sublinear query times like $O(n^{2/3})$. Hence, for sufficiently large n , ϕ , and k , these data structures should outperform trivial solutions that count all colors, or even consider all points. For which values of n , ϕ , and k is this the case, and can we still efficiently construct the data structures for such values of n ?
2. Most of the methods have parameters that govern the space and query time. Theory dictates an optimal choice of these parameters. Does this theoretical optimal choice of parameters correspond to the practical optimal choice?
3. In the analysis of algorithms we often focus only on obtaining worst case guarantees. However, it is unclear whether real world inputs display such worst-case behavior. Instead, the query time may depend on the specific input data. What happens when the colors are very strongly grouped, or very mixed? What happens when the number of colors is small, or very large?

Contributions. Our experiments show that the more complicated data structures indeed outperform more naive methods when k (and n) are large in, on the data sets we performed our tests on. Additionally, in 1D our simpler data structure outperforms the more direct approach of Van der Horst, Löffler, and Staals in query time, build time, and space usage, in practice. Hence, this shows that developing a simpler solution may pay off significantly, despite having the same asymptotical bounds! In terms of the dependency on the parameters,

the results do not deviate much from what one would expect, both on synthetic and on real world data.

Overview and organization. We follow the approach from van der Horst, Löffler, and Staals [16], who observe that chromatic k -NN queries can be answered using two separate queries:

a range finding query: Given a query point q and integer k , find a square Q centered at q that contains exactly k points from P (by finding the k^{th} furthest point $p_k \in P$), and

a mode finding query: Given a query square Q , compute the mode color of $P \cap Q$.

We thus build and implement separate data structures to answer these queries. In subsection 4.2 we briefly describe the data structures for range finding queries. Answering a mode finding query turns out to be much more challenging (and expensive), so the remainder of the article is mostly focused on answering such queries efficiently. In particular, in section 2 we review the data structures of Chan et al. [2] (subsection 2.1) and of van der Horst, Löffler, and Staals [16] (subsection 2.2). In section 3, we show that we can actually use a technique from the cutting-based data structure to analyze the space usage of (a variation of) the grid based data structure, thereby combining the best of both worlds. We present our experimental setup—including the implementation details, the “naive” reference methods, data sets, and the evaluation methods that we use—in section 4. The results of our experiments can be found in section 5. Our code is available at [1].

2 Mode finding. For the second subproblem we are given a query range (an interval in 1D, or a square in 2D), and wish to report the mode color among the points contained in that range. The following (rephrased) observation from Krizanc, Morin, and Smid [9] is central:

LEMMA 2.1 (Krizanc, Morin, and Smid [9]). *Let A and B be multisets. If c is a mode color of $A \cup B$, then either (i) $c \in A$, or (ii) c is a mode of B , or both.*

In Section 2.1 we describe the grid-based method by Chan et al. [2]. In Section 2.2 we describe the cutting based approach of Van der Horst, Löffler, and Staals [16].

2.1 A block based data structure. We briefly review the block-based data structures by Chan et al.. We start with the approach for n points in $\mathbb{R}^1$.

A block mode data structure. Chan et al. give an $O(n)$ space data structure for points in $\mathbb{R}^1$ with

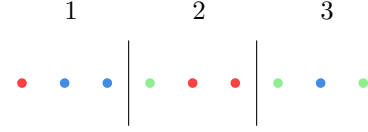


Figure 2.1: An example set of 9 points with $s = 3$ blocks; let r, g, b be the colors red, green, and blue, respectively. Then $\text{blockModes} = \begin{bmatrix} b & r & r \\ - & r & g \\ - & - & g \end{bmatrix}$, $\text{blockModeFrequencies} = \begin{bmatrix} 2 & 3 & 3 \\ - & 2 & 3 \\ - & - & 2 \end{bmatrix}$, $\text{colorIndices}[r] = [0, 4, 5]$, $\text{colorIndices}[g] = [3, 6, 8]$, $\text{colorIndices}[b] = [1, 2, 7]$, and $\text{rankInColor} = [0, 0, 1, 0, 1, 2, 1, 2, 2]$.

$O(\sqrt{n/\log n})$ query time [2]; we will use a simpler version with $O(\sqrt{n})$ query time instead (see Section 3 in [2]). W.l.o.g. we assume here the points have unique integer coordinates in $\{0 \dots n-1\}$. Let $1 \leq s \leq n$ be an integer parameter. We then partition the points into s blocks numbered $1 \dots s$, each containing $O(n/s)$ points. We store four arrays:

blockModes and blockModeFrequencies. For each pair of blocks $1 \leq i \leq j \leq s$, consider the set of points $P_{i,j}$ in blocks i through j . We store a mode color of $P_{i,j}$ as $\text{blockModes}[i,j]$, and its frequency as $\text{blockModeFrequencies}[i,j]$.

colorIndices. For each color c we store a sorted array of the indices of all points of that color $\text{asColorIndices}[c]$.

rankInColor. For every point i we store the rank of that point among its own color as $\text{rankInColor}[i]$.

See Figure 2.1 for an example. Note that, for point i of color c , $\text{colorIndices}[c][\text{rankInColor}[i]] = i$.

The data structure uses $O(n + s^2)$ space, so we set $s = \sqrt{n}$ to use linear space. Construction takes $O(ns)$ time: $O(n)$ time for colorIndices and rankInColor (using a simple scan), and $O(ns)$ time for blockModes (for a given block i we can compute $\text{blockModes}[i,j]$ for all j in $O(n \log n)$ time by scanning from the start of block i to the end of the array, while maintaining the frequency of each color).

Answering a query $[a, b]$. We compute the blocks i and j that contain a and b respectively. If $i = j$ we simply iterate through the block and output the color with the largest frequency. Otherwise, we can split the query in three (possibly empty) parts: the *prefix* from a to the end of block i , the *span* from block $i + 1$ through block $j - 1$, and the *postfix* from the start of block j to b . We first set the candidate mode color $c_m = \text{blockModes}[i+1, j-1]$ and candidate frequency $f = \text{blockModeFrequencies}[i+1, j-1]$

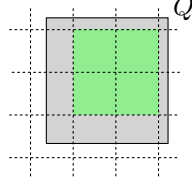


Figure 2.2: A query Q with its span in green, and its generalized pre- and postfix in gray.

to the mode color and frequency of the span. We then iterate through the prefix, from left to right. Consider point p of color c , with $\text{rankInColor}[p] = r$. If we have already considered a point of color c (if $\text{colorIndices}[c][r-1] \geq a$) we skip it. If not, we compute whether color c has a larger frequency than the current candidate mode color: this is the case if and only if $\text{colorIndices}[c][r+f] \leq b$ (i.e. if the next f points of color c all lie within the range $[a, b]$). If so, we set c as the new candidate mode color, and compute its actual frequency by walking through $\text{colorIndices}[c]$ until we leave the range $[a, b]$, starting from $\text{colorIndices}[c][r+f]$. We then proceed to the next point. We similarly process the postfix, in reverse order. By Lemma 2.1 at least one of the considered colors must be a mode color of $[a, b]$. Since each block contains $O(n/s)$ points, this takes $O(n/s)$ time in total.

2.1.1 Extending to higher dimensions. The above idea extends to higher dimensions as well. Let P be a set of n colored points in $\mathbb{R}^d$. We show how to answer range mode queries of the form $Q = [a_1, b_1] \times [a_2, b_2] \times \dots \times [a_d, b_d]$.

In each dimension we again partition the points into s slabs using $s-1$ parallel hyperplanes, such that there are $O(n/s)$ points in any slab. This results in a $s \times s \times s \times \dots$ grid $\mathcal{G}$ with s^d cells. For a query (hyper-)rectangle Q , the *span* generalizes to the the largest grid (hyper-) rectangle contained in Q whose sides lie on grid hyperplanes, and the *pre- and postfix* generalize to the remaining parts of Q . See Figure 2.2.

The data structure. For each distinct span we precompute and store the mode color in a lookup table (accessed by the indices of the minimum and maximum slab containing the span in each dimension). Since there are $O(s^{2d})$ pairs of grid cells and thus $O(s^{2d})$ distinct spans, this uses $O(s^{2d})$ space. Additionally we build a range tree on the points of each color separately, of $O(n \log^{d-1} n)$ total size, for a total size of $O(s^{2d} + n \log^{d-1} n)$.

Answering a query Q . For each dimension, we find the two slabs containing Q 's corners, by binary

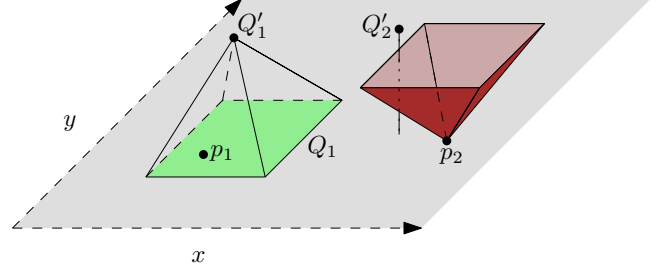


Figure 2.3: A pyramid formed by a square Q_1 , and an upside down pyramid formed by a point p_2 .

searching among the hyperplanes. We iterate through all points in these slabs, and select the subset of them that is also contained in Q . Let C be the set of their colors (similar to the pre- and postfix in 1D). Additionally we look up the mode color c of the span of Q in our table, and add c to C . Then for every color in $c \in C$ we compute the frequency of c in Q using the range tree for color c , and output the color with the highest frequency. By Lemma 2.1, this color must be a mode color of Q . This takes $O(\log n)$ time per color, and since each slab contains $O(n/s)$ colors it takes $O((n/s) \log n)$ time in total.

2.2 A cutting based data structure. We describe the approach of Van der Horst et al. [16] to answer square mode queries in $\mathbb{R}^2$.

Every square Q is defined by a center point (x, y) and a radius r . Hence, a query square Q corresponds to a point $Q' = (x, y, r) \in \mathbb{R}^3$ (i.e. the parameter space $\mathbb{R}^3$ represents all possible queries). See Figure 2.3. We can view this as a “pyramid” with 90° angles whose tip ends in Q' ; the intersection of this pyramid and the (x, y) -plane is exactly Q . A point $p = (p_x, p_y)$ is contained in Q if and only if the point $(p_x, p_y, 0)$ lies below the boundary of the pyramid. Inversely, the set of squares containing p form an “upside down pyramid” ∇_p centered at p . We will consider ∇_p as (the graph of) a function. A square $Q' = (x, y, r)$ contains p if and only if Q' lies above ∇_p . Let $F = \{\nabla_p \mid p \in P\}$ be the set of all these functions. Given a query point Q' (query square Q) we thus wish to find the mode color among all points p whose function ∇_p passes below Q' .

The data structure. A $(1/s)$ -cutting Ξ of F is a subdivision of $\mathbb{R}^3$ into constant complexity cells, so that each cell $\Delta \in \Xi$ is intersected by at most n/s functions from F . Let $F_\Delta \subseteq F$ denote the subset of functions that intersect (the interior of) Δ ; i.e. the *conflict list* of Δ . We compute and store a $(1/s)$ -cutting Ξ of F with $O(s^3)$ cells [3]. For each cell Δ we compute the mode color c_Δ of all functions passing below Δ (and its

frequency). Additionally, we build a data structure that can quickly report the conflict list of a given cell. For every color c we also build a range tree on all points of color c that can answer counting queries. The data structure has size $O(n \log n + s^3)$ and can be built in $O(n \log n + r^3 \log n + nr^2)$ time.

Answering a query $Q' = (x, y, r)$. We first locate the cell Δ containing Q' . By Lemma 2.1, either c_Δ or some color in F_Δ must be a mode color for Q' . Thus, for each of these colors separately we perform a range counting query. Since the conflict list has size $O(n/s)$, this takes $O((n/s) \log n)$ time. We report the color with the highest frequency.

Adaptation for $\mathbb{R}^1$. The above technique for answering mode queries in $\mathbb{R}^2$ can be simplified to work for queries in $\mathbb{R}^1$. Now, a query can be parameterized by midpoint x and radius r , so $\mathbb{R}^2$ represents all possible queries $Q = (x, r)$. The function ∇_p of a point p is now “v-shaped”, and there exists a cutting of all these functions with $O(s^2)$ cells. Answering a query is analogous to the version in $\mathbb{R}^2$.

3 The grid data structure for range mode queries with L_∞ disks. In this section we first present a data structure that can answer queries under the L_∞ distance metric, by combining ideas from both the block-based and the cutting-based methods. We later show how to extend this to other metrics such as L_1 .

The grid based data structure from subsection 2.1 can answer queries with arbitrary axis-aligned orthogonal boxes. However, to answer chromatic k -NN queries in the L_∞ distance, it suffices to answer queries whose boxes that have equal side-lengths; that is, hypercubes. Although there are $\Theta(s^{2d})$ possible combinatorially distinct rectangular spans (and thus the blocks data structure uses $\Theta(n \log^{d-1} n + s^{2d})$ space), not all of these can actually be the span of a hypercube query; see Figure 3.1. We formalise this below.

Let a *square span* be such a grid-rectangle whose sides lie on grid hyperplanes that can be the span of a hypercube. Next, we will show that only $O(s^{d+1})$ such square spans exist, by borrowing some of the ideas that were used in the data structure of subsection 2.2. Hence, we will compute and store the mode color only for spans that correspond to hypercubes (using e.g. a binary search tree or a hash table). This thus reduces the space used by the data structure to $O(n \log^{d-1} n + s^{d+1})$. Furthermore, we will argue we can construct the data structure in $O(n \log^{d-1} n + ns^d)$ time. Hence, this version of the grid based method matches the performance of the more complicated cutting-based structure from subsection 2.2; in fact, it can be seen as an alternate (and in our opinion simpler) description of the same data structure.

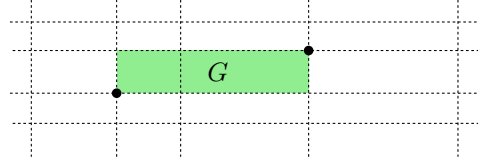


Figure 3.1: There is no square for which G is the largest grid-rectangle contained in it.

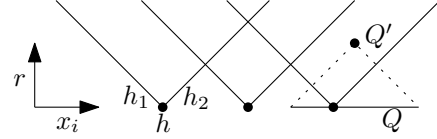


Figure 3.2: A d -dimensional grid hyperplane $h : x_i = a$ creates two $(d+1)$ -dimensional hyperplanes h_1, h_2 . A square Q intersects h if and only if its corresponding point Q' lies above h_1 and h_2 .

Reducing the space usage. Consider the grid $\mathcal{G}$ from subsection 2.1 that partitions $\mathbb{R}^d$ into s^d grid cells using $d(s-1)$ hyperplanes. As in Section 2.2 we embed this grid on the plane $r = 0$ in a $d+1$ -dimensional space (where r denotes the additional $(d+1^{\text{th}})$ coordinate). For each d -dimensional grid hyperplane $h : x_i = a$ we create two $(d+1)$ -dimensional hyperplanes $h_1 : r = a - x_i$ and $h_2 : r = -a + x_i$. See Figure 3.2 for an illustration. Let H be the set of all $2d(s-1)$ of these $(d+1)$ -dimensional hyperplanes that we create this way. A square Q centered at $(Q_1, \dots, Q_d)$ with radius Q_r intersects the grid hyperplane h if and only if $Q_i + Q_r \geq a$ and $Q_i - Q_r \leq a$, which happens if and only if $Q' = (Q_1, \dots, Q_d, Q_r)$ lies above both h_1 and h_2 .

Consider the arrangement $\mathcal{A}$ of all $O(s)$ hyperplanes in H , which has complexity $O(s^{d+1})$ [5]. All $(d+1)$ -dimensional points within one cell of the arrangement are indistinguishable w.r.t. the hyperplanes in H . That means their corresponding squares intersect the same grid-hyperplanes, and thus have the same span. It follows that there are only $O(s^{d+1})$ distinct spans of square queries in the grid.

Preprocessing. Each cell in the arrangement $\mathcal{A}$ corresponds to a d -dimensional square span. Consider two adjacent cells in $\mathcal{A}$, and observe that they are combinatorially equivalent w.r.t. the $d+1$ -dimensional hyperplanes, except the one dividing them. It follows that their corresponding spans are equivalent w.r.t. the d -dimensional hyperplanes defining the grid, except for at most one slab in one dimension.

Hence, we will traverse the cells of $\mathcal{A}$ (using e.g. a depth first search) while maintaining the frequency of every color in the span corresponding to the current cell.

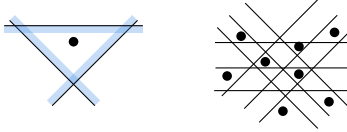


Figure 3.3: A triangle unit disk in $\mathbb{R}^2$ that is the intersection of three halfspaces, and the irregular grid based on this unit disk where each strip contains two points.

After every step we store the current mode color with this current slab in the lookup table. Since two adjacent spans differ by at most one slab, which contains $O(n/s)$ points, we can update these frequencies in $O(n/s)$ time per step (see Section 4.3 in [16] for why these updates can be performed in $O(1)$ time per step), or $O(ns^d)$ time in total for all $O(s^{d+1})$ cells. Note that it is not even needed to explicitly construct $\mathcal{A}$ as it itself has a grid-like structure: all the h_1 hyperplanes for the grid lines of some coordinate x_i are parallel, and appear in the same order as the grid hyperplanes. Hence, it suffices to compute these $2d$ sorted lists of these hyperplanes, and keep track of the indices corresponding to the current cell of $\mathcal{A}$.

Other distance metrics. The above technique can also be used for other distance metrics whose unit disk is a constant-complexity convex polytope, for example L_1 . The main idea is that we can still define an irregular grid, and argue that the number of grid cells that define queries is small. See Figure 3.3 for an illustration, and see section 7 for the proof. In particular, we obtain the following result:

THEOREM 3.1. *Let $\hat{L}$ be a distance metric whose unit disk is a constant-complexity convex polytope. Given a set of points in $\mathbb{R}^d$ and a parameter s we can build a data structure of size $O(n \log^{d-1} n + s^{d+1})$ in $O(n \log^{d-1} n + ns^d)$ time. Given a query point $q \in \mathbb{R}^d$ and radius $r \in \mathbb{R}$ it can report the mode color of all points at distance at most r from q , according to $\hat{L}$, in $O((n/s) \log^{d-1} n)$ time.*

This bound matches that of Van der Horst, Löffler, and Staals; specifically, choosing $s = (n \log^{d-1} n)^{1/(d+1)}$ yields an $O(n \log^{d-1} n)$ space datastructure with query time $O((n \log^{d-1} n)^{1-1/(d+1)})$ [16] (see Lemma 23, and Theorem 45). Similar to Van der Horst, Löffler, and Staals, we can also adapt this to a version using $O(n)$ space (by using high fanout range trees), using slightly more preprocessing and query time; see their article for details.

4 Experimental setup. We experimentally test and compare several data structures for mode find-

ing. The data structures were implemented in C++. The code and experimental results are available at [1]. **TODO: publish code properly**

To answer 2D mode finding queries, the cutting-based datastructure from subsection 2.2 would require building 3D arrangements with point location. To our knowledge, these are unfortunately not readily available in any widely used computational geometry library (and would be a large amount of work to implement from scratch). Instead, we implement the cutting-based datastructure for 1D queries; this uses 2D arrangements, which are supported by The Computational Geometry Algorithms Library (CGAL) [15, 17]. We compare the 1D cutting-based structure to other 1D mode finding datastructures. If the cutting-based structure performs worse than the alternatives even in 1D, there is little hope that it will do better in higher dimensions.

4.1 Implemented mode finding data structures. See Table 4.1 for an overview of the worst case space usage and query times.

Both in 1D and in 2D we implement the following data structures:

- 1D/2D-report** report all points within the query square using a range tree, and iterate through them while maintaining counts.
- 1D/2D-rangeTrees** query the frequency of all ϕ colors separately (using a range tree per color).
- 1D/2D-grid** the data structure described as described in section 3

In 1D we have additionally implemented the following data structures:

- 1D-blocks** using the approach of Chan et al. [2] described in subsection 2.1.
- 1D-cutting** using the approach of van der Horst [16] described in subsection 2.2.

4.2 Implemented range finding data structures. Although the focus of this research is on the structures for mode finding queries, a full k NN query also consists of a range finding query, where we are given a query point q and an integer k , and wish to compute the radius of a square centered at q that contains exactly k points. Van der Horst, Löffler, and Staals [16] present a structure in 1D with $O(\log n)$ query time using $O(n)$ space, and in 2D with $O(\log^2 n)$ query time using $O(n \log n)$ space. For simplicity, we instead implement the $O(\log^2 n)$ technique both for 1D and 2D queries, since the range finding queries are much faster

Structure	Space	Query time
1D report	$O(n)$	$O(\log n + k)$
1D rangeTrees	$O(n)$	$O(\phi \log n)$
1D blocks	$O(n + s^2)$	$O(n/s)$
1D cutting	$O(n \log n + s^2)$	$O((n/s) \log n)$
1D grid	$O(n \log n + s^2)$	$O((n/s) \log n)$
2D report	$O(n \log n)$	$O(\log^2 n + k)$
2D rangeTrees	$O(n \log n)$	$O(\phi \log^2 n)$
2D grid	$O(n \log n + s^3)$	$O((n/s) \log^2 n)$

Table 4.1: The worst case space usage and query times of the different implemented methods.

than the mode finding queries anyway. We describe the technique for 2D; it trivially applies to 1D as well.

We build a range tree on all points that can answer rectangle counting queries. Consider all points with smaller x -coordinate than q ; we will perform a binary search on these points. Given the point p with median x -coordinate among these points, we construct the square centered at q with radius $|q_x - p_x|$, and query the number of points contained in this square. If this square contains less than k points we are looking for a larger square, so we recurse on the points left of p (= further away from q). If the square contains more than k points we recurse on the points right of p (= closer to q). If the square contains exactly k points we are done. If this binary search terminates and we have not yet found a square containing exactly k points we repeat a similar procedure on all points with larger x -coordinate, and then do the same for the y -coordinates. This uses $O(n \log n)$ space for the range tree, and a query is answered using $O(\log n)$ range-counting queries, in $O(\log^2 n)$ total time.

4.3 Measurements. For each datastructure we measure the space usage, the preprocessing time, and the average query time. We measure these quantities as follows.

Preprocessing and query time. We measure time using `std::chrono::high_resolution_clock`, just before and after building the datastructure or performing the query. This reports so called “wall-clock” time in nanoseconds (10^{-9} seconds). For queries, we first perform 1000 and discard their result (since the earliest queries might be slightly slower due to caching) and then take the average time over 10000 queries to minimize fluctuations and randomness.

Space usage. We measure space “manually”, by counting the number of bytes required to store all

the primitive types present in a data structure. For example, a vector that stores 10 ints of 4 bytes each uses $10 * 4 = 40$ bytes of space. Additionally, the `cutting` data structure contains a more complicated data structure, namely the `CGAL::Arrangement_2` (and accompanying `CGAL::Arr_trapezoid_ric_point_location`) object. For an arrangement with v vertices, e edges, and f faces we estimate the space usage assuming that it is implemented using a very basic DCEL, containing three edge-pointers per edge (next, twin, previous), and one edge pointer per vertex and face. The space usage computed this way can be an under-estimate of the real space usage, as we ignore any overhead such as allocated but unused space, or auxiliary pointers. This mostly affects the `cutting` structure, which may thus use more space in practice. However, as we will see in Section 5, even with this under-estimate the `cutting` structure uses *more* space than the other data structures considered.

4.4 Data sets. We use the following data sets in our evaluation.

Synthetic uniform random data. Given parameters n and ϕ , we generate n points, uniformly sampled within a 10000-radius cube centered at the origin, each with a color from $1 \dots \phi$, also uniformly at random. See Figure 4.1.

Synthetic grouped data. Given parameters n , ϕ , $\alpha \in [0, 1]$, and $\gamma \in \{0 \dots n\}$, we generate n points, uniformly sampled within a cube. We pick γ of them as *center points*, and give them a color from $1 \dots \phi$, uniformly at random. For each of the remaining points, with probability α we set its color to that of the closest center point, and with probability $1 - \alpha$ we choose a color uniformly at random. The larger the value of α , the stronger the grouping in the produced coloring. Specifically, with $\alpha = 0$ this is equivalent to the uniform random coloring, and with $\alpha = 1$ the point set will be colored according to the voronoi diagram of the centres. See Figure 4.1.

Real data. We created point sets by discretizing map data from OpenStreetMap (OSM) [11] using QGIS [12]. On these maps different terrain-types have been marked by different colors (e.g. road, building, grass, etc.), see Figure 4.2. We discretize this map into a colored point set by sampling it in a grid pattern (the OSM dataset), and use sub-sampling to obtain smaller subsets of the same dataset.

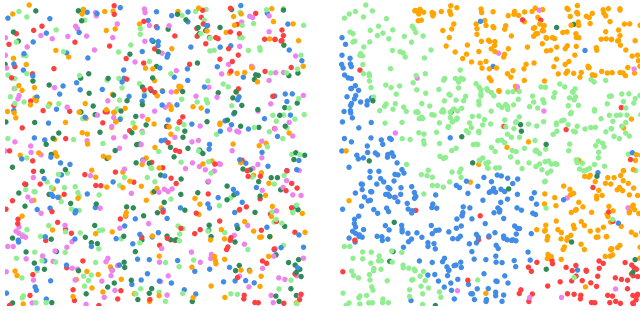


Figure 4.1: Left: a set of 10 000 points with 6 uniformly random colors. Right: a set of 1000 points with grouped colors, with $\alpha = 0.9$ and $\gamma = 10$.

The set of experiments run in 1D is shown in Table 4.2, and in 2D is shown in Table 4.3. For each scenario we build every datastructure from subsection 4.1 (when applicable). Since the `cutting` structure turned out to be significantly slower than the others, we use values of n , ϕ , s , and γ that are up to 10 times lower than for the other structures.

4.5 Queries. A query is a square defined by a center point $c \in \mathbb{R}^2$, and a value $k \in 1..n$ specifying its size (in the number of points that will be in the query square). We use a fixed set of possible k values, ranging from 1 to n . For each experiment and each value of k we produce 10000 centers; for the synthetic datasets we generate these uniformly at random (within the same bounds as the point set itself), for the real datasets we randomly sample (and remove) 10000 points from the input point set. Given such a center c we then compute the smallest square centered at c containing exactly k points using the range finding data structure, and then compute a mode color using one of the mode finding data structures.

5 Experimental results and analysis. In this section we show the most interesting results of our experiments, and analyze their meaning with respect to our research questions. All results can be found along with the source code in [1]. [ERWIN SAYS: and in appendix?](#)

5.1 Practicality of sublinear methods. Our first research question concerns how practical these sublinear methods are. We first compare the `cutting` structure to the `grid` and `blocks` structures in 1D, and then compare the `grid` method to the naive `report` and `rangeTrees` methods in both 1D and 2D. We then study the dependency of the different structures to the parameter k , and lastly consider the relative importance of the range finding query versus the mode finding query.

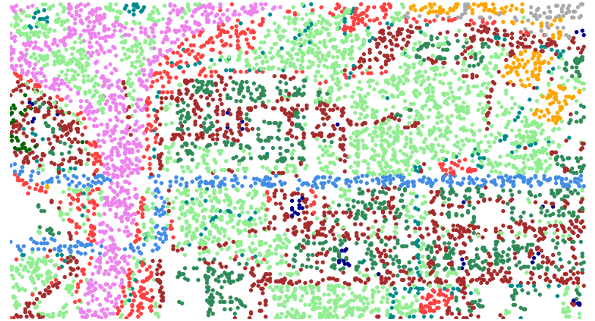
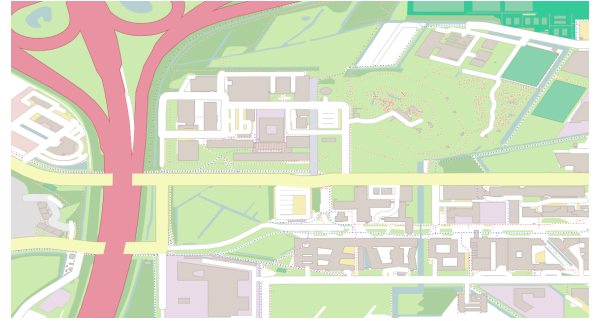


Figure 4.2: A map colored by terrain-type, then discretized (the actual colors after discretization are randomized).

Cuttings in practice. See Figure 5.1. The `grid` structure outperforms the `cutting` structure in query time by a around factor 2, and in build time and space usage by around a factor 10, despite having the same asymptotical bounds and essentially doing the same thing. This indicates that the simplicity of the `grid` structure, i.e. avoiding to build an explicit 2D arrangement and a point-location structure on it, indeed provides improved performance. Anectotally but importantly, the `grid` structure was also significantly simpler to implement than the `cutting` structure.

The `blocks` structure vastly outperforms all other methods in query time, most notably for high k values. This shows that avoiding the range counting queries (and thus dealing with each point in amortized constant time) indeed pays off significantly. Unfortunately these specific techniques do not easily translate to higher dimensions.

Comparing grid and naive methods. See Figure 5.1 for the 1D results and Figure 5.2 for the 2D results. In both cases, for the chosen parameters, the `grid` structure has a lower query time than both the `report` and the `rangeTrees` structures, showing that the more complicated sub-linear methods can indeed provide better performance than the naive methods, while using slightly more space and significantly more build time.

Take for example $n = 500\,000$ in 2D, where the `grid`

Experiment	n	ϕ	s	γ	α
Uniform data with increasing n	25 000 - 300 000	25 000	$\sqrt{n}$	-	-
Uniform data with increasing ϕ	200 000	1 - 200 000	$\sqrt{n}$	-	-
Uniform data with increasing s	200 000	25 000	2 - 1000	-	0 - 1
Grouped data with increasing α	200 000	25 000	$\sqrt{n}$	10 000	-

Table 4.2: The parameters for the 1D experiments.

Experiment	n	ϕ	s	γ	α
Uniform data with increasing n	10 000 - 150 000	10 000	$\sqrt{n}$	-	-
Uniform data with increasing ϕ	100 000	1 - 100 000	$\sqrt{n}$	-	-
Uniform data with increasing s	100 000	10 000	2 - 1000	-	-
Grouped data with increasing α	100 000	10 000	$\sqrt{n}$	5000	0-1
Real data: OSM	5000 - 8000	10	$\sqrt{n}$	-	-

Table 4.3: The parameters for the 2D experiments.

structure has around query time 4 ms and build time 900 000 ms, while the **rangeTrees** structure has around query time 20 ms and build time 300 ms. This means that, at these parameters, the **grid** structure will start outperforming the **rangeTrees** structure from around 50 000 queries, or only around 10% of the number of points.

(In)dependence of k One of the main theoretical motivations behind the design of the **blocks**, **cutting**, and **grid** structures was to have a query time independent of k and ϕ , in contrast to the simple **report** and **rangeTrees** structures. Although it is true that the asymptotic worst-case upper bound is independent of k , in practice some dependency remains visible for the **grid** structure. See Figures 5.3 and 5.4. The smaller the value of k , the fewer points a query q contains, and thus the fewer colors in the slabs containing q 's cornerpoints are contained in q . This results in fewer range-searches to be done, and thus a lower query time.

For k close to n ($k > 350\,000$ in Figure 5.4) the query time starts decreasing as k increases. This behaviour can be seen more strongly for the **rangeTrees** structure. This is because very large queries are likely to contain all points of a color, in which case a query is answered much more quickly, both due to having a smaller set of canonical nodes and due to caching.

Ratio between range-finding and mode-finding Our main focus has been on the different structures for mode-finding queries given a square, but to answer a k NN query we first need to actually find this square using a range-finding structure. See Figure 5.2. The range query contributes only a small portion of the total query time, being at least 10 times as fast as all mode-finding structures. This indicates that, both in

theory and in practice, the mode-finding portion of the problem requires significantly more time, and is thus more interesting to optimize.

5.2 Parameter choice. Our second research question considers the practical influence of the user-set parameter s . See Figure 5.6, where $n = 100\,000$ and $\phi = 1000$ ($n = 10\,000$ and $\phi = 100$ for the **cutting** structure). In the figure we separated the s -dependent space usage (most importantly the **blockModes** table) from the s -independent space usage (the range trees).

Theory tells us we should choose $s = O(n^{1/d+1})$ for a good tradeoff between (linear) space and (sub-linear) query time, but this leaves us free to choose a constant ($s = \frac{n^{1/d+1}}{10}$ vs $s = 10n^{1/d+1}$). A logical choice would be to choose s such that the space usage dependent on s is about equal to the space usage independent from s , to obtain a good query time without sacrificing too much extra space. From our experiments, this is somewhere between $s = 75$ and $s = 100$, which is quite close to the theoretical value $s = n^{1/d+1} = 63$. This indicates that the choice of $s = n^{1/d+1}$ indeed provides a good practical tradeoff between query time and space usage.

5.3 Dependency on data distribution. Our last research question considers the dependency of the **grid** structure on the color distribution of the data. We compare the performance on uniform random point sets with varying numbers of total colors ϕ , on uniform random point sets with varying level of groupedness α , and on real datasets.

Varying ϕ . See Figure 5.5. The query times are very fast for very low values of ϕ , since the query time is mainly dominated by the number of counting queries that must be done; if there are only few colors, we must

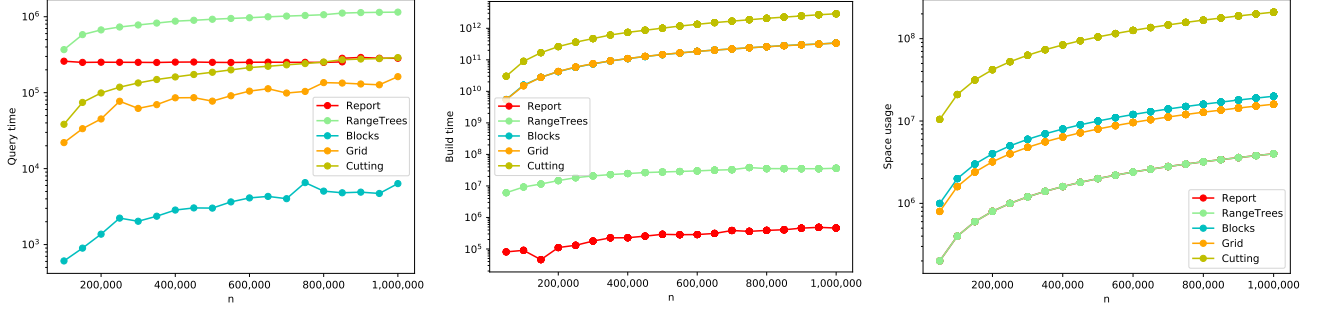


Figure 5.1: Query time, build time, and space usage of the five 1D methods, using $k = 100\,000$ and $\phi = 50\,000$.

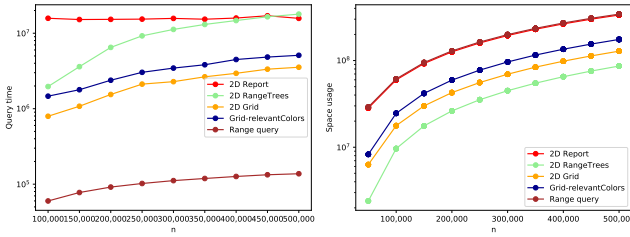


Figure 5.2: Query time and space usage for the 2D structures and the range-finding structure, using $k = 100\,000$ and $\phi = 50\,000$.

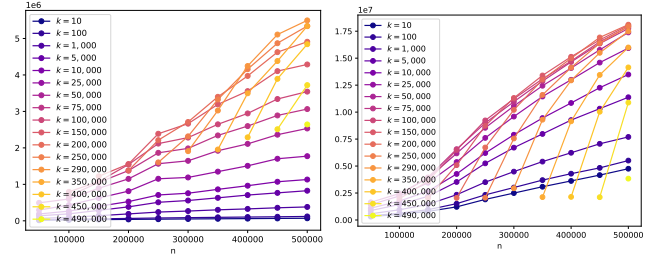


Figure 5.3: Query time **grid** structure (left) and the **rangeTrees** structure (right) for varying k values, using $\phi = 50\,000$.

only perform few counting queries. As ϕ grows larger we see a slight decrease in query time, since the number of points per color decreases, and thus each individual counting query becomes faster. Similar behavior can be observed in the **rangeTrees** structure. The effect is more pronounced for the **grid** structure since it does at most $O(n/s)$ counting queries, rather than ϕ counting queries for the **rangeTrees** structure.

Varying α . See Figure 5.7. As α increases, the query time decreases significantly. This is likely partially because there are fewer different colors in each strip and we thus collect a smaller set of colors for which we have to perform counting queries. However, similar behavior can again be observed in the **rangeTrees** structure. This might be because, if a color is very strongly grouped, many queries contain the same subset of points, for example containing all points of that color or none at all, increasing cache-effectiveness. In contrast, if a color is distributed uniformly, it is much less likely that two queries contain the same points of that color.

Real datasets. See Figure 5.8, and note that these datasets use only $\phi = 11$ colors, and thus all structures have relatively low query time for reasons discussed above. The performance on all three datasets is nearly identical for all values of n and k , indicating that with so few colors, it does not make much difference

whether they are grouped realistically, grouped synthetically, or colored uniformly at random.

6 Concluding remarks. In this article we implemented and compared several different data structures for mode finding queries. We found that our simplified **grid** structure performed better than the **cutting** structure in 1D, despite having equal worst-case asymptotic lower bounds, indicating that it may be worthwhile to strive for simplicity in algorithm design, both for ease of understanding and for performance reasons. Both in 1D and in 2D the **grid** structures could also outperform the naive **rangeTrees** and **report** structures, for certain parameters ϕ and k , already for a modest number of queries. Secondly, the dependency on the parameter s was as we would expect. Lastly, the **grid** structures was relatively consistent across different color distributions in the input set; performing slightly better on datasets with very few or very many colors and on datasets with highly grouped colors. **ERWIN SAYS: nog iets van algemene conclusie? weet niet precies wat**

Future work. In this article we focused on the L_∞ distance measured, and showed that we could use its geometric properties to significantly simplify the cutting-based mode finding data structure. Our results extend to other polygonal distance functions (such as L_1).

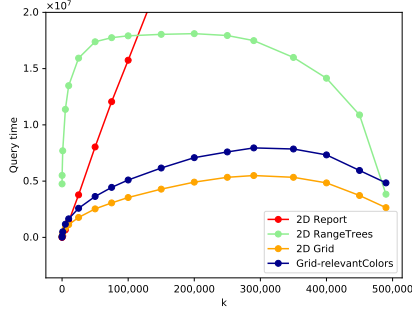


Figure 5.4: Query time of the **grid** structure as a function of k , using $n = 500\,000$ and $\phi = 50\,000$. The **report** graph is simply a straight line, and has been cut.

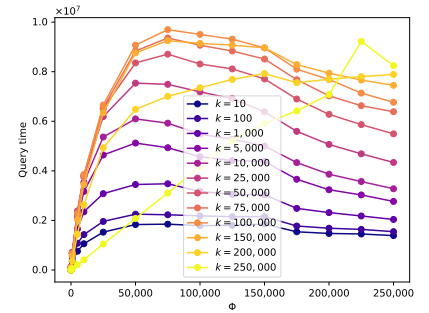
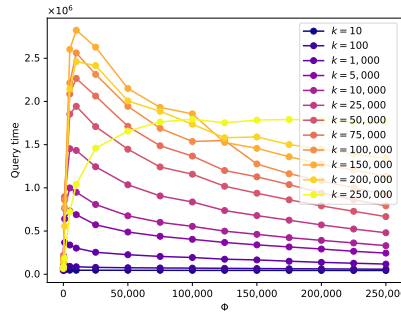


Figure 5.5: Query time of the **grid** structure (left) and the **rangeTrees** structure (right) as a function of ϕ , using $n = 250\,000$.

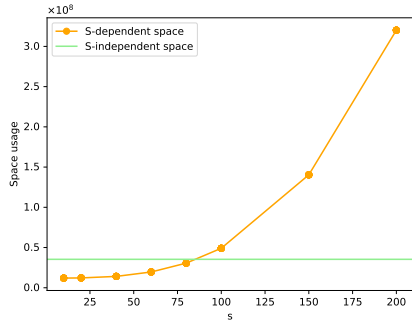


Figure 5.6: Space usage of the **grid** structure as a function of s , using $n = 250\,000$ and $\phi = 50\,000$, split into s -dependent and s -independent space.

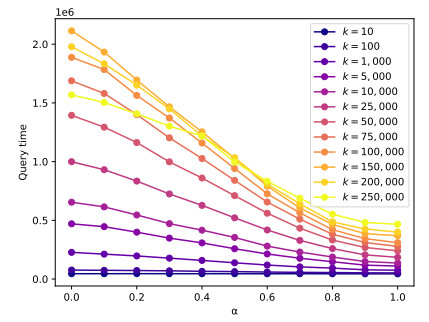
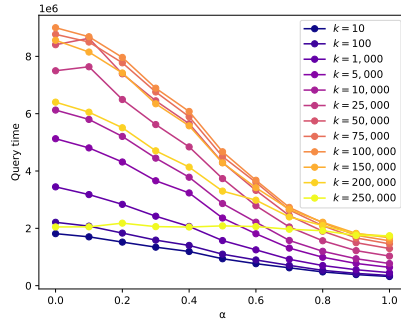


Figure 5.7: Query time of the **grid** structure (left) and the **rangeTrees** structure (right) as a function of α , using $n = 250\,000$ and $\phi = 50\,000$.

The cutting based approach works for even more general distance measures (e.g. L_2 as well). An interesting question is if a similar simplifications are applicable in that setting as well. Furthermore, how does this more general method perform in practice?

References

- [1] *Code to generate / run experiments (anonymized)*. <https://anonymous.4open.science/r/anonymousExperimentalCode-F2D0>.
- [2] T. M. CHAN, S. DUROCHER, K. G. LARSEN, J. MORRISON, AND B. T. WILKINSON, *Linear-space data structures for range mode query in arrays*, Theory of Computing Systems, 55 (2014), pp. 719–741.
- [3] B. CHAZELLE, *Cutting hyperplanes for divide-and-conquer*, Discrete & Computational Geometry, 9 (1993), pp. 145–158.
- [4] T. COVER AND P. HART, *Nearest neighbor pattern classification*, IEEE transactions on information theory, 13 (1967), pp. 21–27, <https://doi.org/10.1109/TIT.1967.1053964>, <https://doi.org/10.1109/TIT.1967.1053964>.
- [5] H. EDELSBRUNNER, J. O’ROURKE, AND R. SEIDEL, *Constructing arrangements of lines and hyperplanes with applications*, SIAM J. Comput., 15 (1986), pp. 341–363, <https://doi.org/10.1137/0215024>, <https://doi.org/10.1137/0215024>.

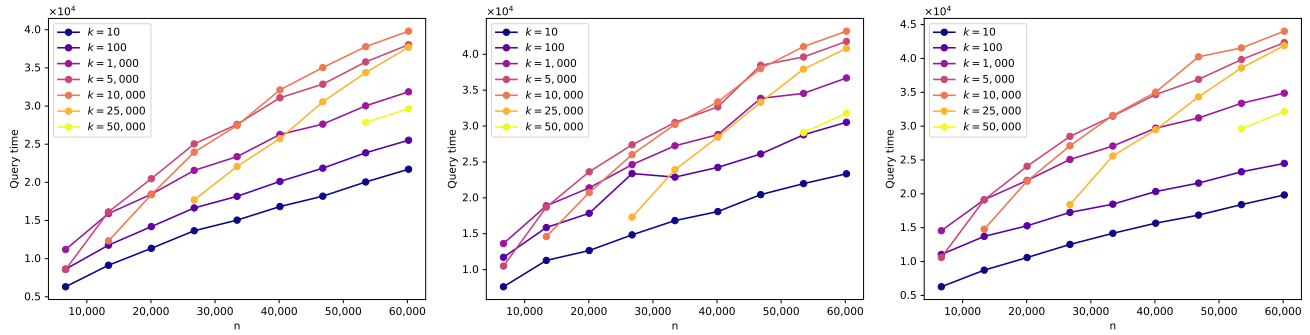


Figure 5.8: Query time of the **grid** structure in 2D on subsets of a real dataset (OSM-bbg) with $n = 66\,870$ and $\phi = 11$ (left), and on synthetic datasets with uniform random colors (middle), and on synthetic datasets with grouped colors with $\alpha = 1$ and $\gamma = 1000$ (right).

- [6] P. GUPTA, R. JANARDAN, S. RAHUL, AND M. SMID, *Computational geometry: Generalized (or colored) intersection searching*, in Handbook of Data Structures and Applications, Chapman and Hall/CRC, 2004, pp. 1043–1058, <https://doi.org/https://doi.org/10.1201/9781420035179>.
- [7] P. GUPTA, R. JANARDAN, AND M. H. M. SMID, *Further results on generalized intersection searching problems: Counting, reporting, and dynamization*, J. Algorithms, 19 (1995), pp. 282–317, <https://doi.org/10.1006/JAGM.1995.1038>, <https://doi.org/10.1006/jagm.1995.1038>.
- [8] R. JANARDAN AND M. A. LÓPEZ, *Generalized intersection searching problems*, Int. J. Comput. Geom. Appl., 3 (1993), pp. 39–69, <https://doi.org/10.1142/S021819599300004X>.
- [9] D. KRIZANC, P. MORIN, AND M. SMID, *Range mode and range median queries on lists and trees*, Nordic Journal of Computing, 12 (2005), pp. 1–17.
- [10] J. MISRA AND D. GRIES, *Finding repeated elements*, Science of computer programming, 2 (1982), pp. 143–152.
- [11] OPENSTREETMAP CONTRIBUTORS, *Planet dump retrieved from <https://planet.osm.org>*. <https://www.openstreetmap.org>, 2017.
- [12] QGIS DEVELOPMENT TEAM, *QGIS Geographic Information System*, QGIS Association, 2026, <https://www.qgis.org>.
- [13] S. RAHUL, P. GUPTA, R. JANARDAN, AND K. S. RAJAN, *Efficient top-k queries for orthogonal ranges*, in International Workshop on Algorithms and Computation, Springer, 2011, pp. 110–121.
- [14] J. TANG, S. ALELYANI, AND H. LIU, *Data classification: algorithms and applications*, in Data Mining and Knowledge Discovery Series, CRC press, 2014, pp. 37–64.
- [15] THE CGAL PROJECT, *CGAL User and Reference Manual*, CGAL Editorial Board, 6.1.1 ed., 2026, <https://doc.cgal.org/6.1.1/Manual/packages.html>.
- [16] T. VAN DER HORST, M. LÖFFLER, AND F. STAALS, *Chromatic k-nearest neighbor queries*, Journal of Computational Geometry, 16 (2025), pp. 1–43, <https://doi.org/10.20382/jocg.v16i1a3>, <https://doi.org/10.20382/jocg.v16i1a3>.
- [17] R. WEIN, E. BERBERICH, E. FOGEL, D. HALPERIN, M. HEMMER, O. SALZMAN, AND B. ZUKERMAN, *2D arrangements*, in CGAL User and Reference Manual, CGAL Editorial Board, 6.1.1 ed., 2026, <https://doc.cgal.org/6.1.1/Manual/packages.html#PkgArrangementOnSurface2>.
- [18] A. YU, P. K. AGARWAL, AND J. YANG, *Processing a large number of continuous preference top-k queries*, in Proceedings of the 2012 ACM SIGMOD international conference on management of data, 2012, pp. 397–408.

7 Omitted Proofs.

THEOREM 3.1. *Let $\hat{L}$ be a distance metric whose unit disk is a constant-complexity convex polytope. Given a set of points in $\mathbb{R}^d$ and a parameter s we can build a data structure of size $O(n \log^{d-1} n + s^{d+1})$ in $O(n \log^{d-1} n + ns^d)$ time. Given a query point $q \in \mathbb{R}^d$ and radius $r \in \mathbb{R}$ it can report the mode color of all points at distance at most r from q , according to $\hat{L}$, in $O((n/s) \log^{d-1} n)$ time.*

Proof. The unit disk in the $\hat{L}$ metric is the intersection of some constant number c of d -dimensional halfspaces, bounded by $(d-1)$ -dimensional hyperplanes $h_1 \dots h_c$.

Consider one hyperplane h_i of the unit polytope. We sort the points in the direction of its normal, and partition space into s strips using s $(d-1)$ -dimensional hyperplanes parallel to h_i , each strip containing $O(n/s)$ points. Doing this for all c polytope hyperplanes h_i results in a set H of $O(s)$ strip-hyperplanes in total (recall c is a constant), forming a non-standard grid, as in Figure 3.3. Given a query Q we can once again find the span, the largest sub-grid that is fully contained in Q , of which we wish to know the mode color. There are $O(s^c)$ such distinct possible spans, since there are $O(s)$ strips in all c directions. We once again show that there are only $O(s^{d+1})$ distinct spans that correspond to actual queries. The rest of the data structure remains the same; after finding the mode of a span we iterate through all remaining points in the strips containing the sides of Q .

Consider again the parameter space $\mathbb{R}^{d+1}$ where each point $(x, r) \in \mathbb{R}^{d+1}$ corresponds to a query centered at $x \in \mathbb{R}^d$ with radius $r \in \mathbb{R}$, i.e. the unit polytope scaled by a factor r . Consider the unit polytope at height $r = 1$ centered at the origin, and for each d -dimensional halfspace bounded by $(d-1)$ -dimensional hyperplane h_i that the polytope consists of, consider the $(d+1)$ -dimensional halfspace bounded by d -dimensional hyperplane h'_i that goes through it and through the origin. The intersection of these halfspaces h'_i is exactly the set of points (x, r) corresponding to a query centered at x with radius r that contains the origin. This is analogous to the upside-down pyramid from Figure 2.3, where the unit polytope is a square.

For each $(d-1)$ dimensional grid-hyperplane in H we create the d -dimensional hyperplane through it that is parallel to h'_i . These $O(s)$ hyperplanes partition $\mathbb{R}^{d+1}$ into $O(s^{d+1})$ cells. As before, all $(d+1)$ -dimensional points within one cell of the arrangement are indistinguishable w.r.t. the d -dimensional hyperplanes. That means their corresponding queries intersect the same grid-hyperplanes from H , and thus have the same span.

It follows that there are only $O(s^{d+1})$ distinct spans that correspond to actual queries in the grid. Precomputation of these spans remains the same, by traversing the above arrangement (implicitly). $\square$